

# Spring Security: AuthenticationManager, AuthenticationProvider и работа с документацией

Исчерпывающий разбор архитектуры диспетчеризации аутентификации, анатомии провайдеров, экспорта бинов и методов эффективного чтения официальной документации Spring ([docs.spring.io](https://docs.spring.io)) на английском языке.

**Контекст разработки:** Java 17, Spring Boot 3.5.5, Spring Security 6.x.

**Для кого:** Разработчики, переходящие в Java/Spring (включая специалистов из 1C/Enterprise), желающие досконально понимать внутреннее устройство фреймворка без «магии».

**Ключевые темы:** Цепочка проверки подлинности, `ProviderManager`, `DaoAuthenticationProvider`, `AuthenticationConfiguration`, устранение `NoSuchBeanDefinitionException`, навигация по [docs.spring.io](https://docs.spring.io), чтение исходников в IntelliJ IDEA, 4-шаговая методика работы с английской документацией (A2+/B1).

## Содержание конспекта

---

1. Интуитивная аналогия: Бюро пропусков и экспертные отделы
2. Что такое AuthenticationManager и ProviderManager под капотом
3. Что такое AuthenticationProvider и какие виды бывают
4. Пошаговая анатомия работы DaoAuthenticationProvider (5 этапов)
5. Сквозная блок-схема процесса аутентификации (от HTTP до БД)
6. Зачем пишется отдельный метод @Bean для AuthenticationManager
7. Два канонических способа объявления бина в Spring Security 6
8. Что скрывается внутри AuthenticationConfiguration.java
9. Где искать в официальной документации Spring (docs.spring.io)
10. Как смотреть документацию и исходники прямо в IntelliJ IDEA
11. Как комфортно читать документацию на английском с уровнем A2+/B1
12. Микро-словарь терминов Spring Security (16 ключевых понятий)

## 1. Интуитивная аналогия: Бюро пропусков и экспертные отделы

---

Чтобы навсегда устранить путаницу между десятком похожих названий ( `AuthenticationManager` , `AuthenticationProvider` , `UserDetailsService` , `UserDetails` ), представим организацию службы безопасности крупного предприятия:

| Компонент Spring Security       | Аналогия из реальной жизни         | Обязанности и зона ответственности                                                                                                                                   |
|---------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AuthenticationManager           | Директор бюро пропусков            | Лично не проверяет подлинность водяных знаков и не копается в архивах. Координирует процесс: принимает документ посетителя и передает его в нужный экспертный отдел. |
| AuthenticationProvider          | Экспертный отдел / Специалист      | Узкоспециализированный эксперт по конкретному типу документов. Умеет проверить паспорт с фото, электронную смарт-карту или отпечаток пальца.                         |
| DaoAuthenticationProvider       | Эксперт по связке «Логин + Пароль» | Специалист, знающий регламент: «Сходи в картотеку кадров по имени сотрудника, достань досье и сравни отпечаток пароля на криптографическом приборе».                 |
| UserDetailsService              | Архивариус картотеки               | Знает, в каком шкафу (в какой таблице PostgreSQL) лежит досье сотрудника. Достает карточку по фамилии/логину. <b>Пароли сам не проверяет</b> — просто выдает данные. |
| UserDetails<br>( AppPrincipal ) | Личное дело / Карточка сотрудника  | Контейнер данных: хэш пароля из базы, список допусков и прав ( Authorities ), флаги активности и блокировки.                                                         |
| PasswordEncoder                 | Криптографический прибор           | Математический калькулятор. Сверяет введенный пароль с захешированным паролем из базы:<br><code>matches(raw, hash)</code> .                                          |
| SecurityContext                 | Журнал дежурного / Временный бейдж | Хранит информацию о том, кто прямо сейчас находится в здании и выполняет текущий HTTP-запрос.                                                                        |

## 2. Что такое AuthenticationManager и ProviderManager под капотом

В исходном коде Spring Security интерфейс `AuthenticationManager` содержит ровно один метод:

```
package org.springframework.security.authentication;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;

public interface AuthenticationManager {
    Authentication authenticate(Authentication authentication) throws AuthenticationException;
}
```

- **На вход:** Неаутентифицированный объект `Authentication` (например, `UsernamePasswordAuthenticationToken` с сырыми логином и паролем, у которого флаг `isAuthenticated() == false`).
- **На выходе:** Подтвержденный объект `Authentication` (с `Principal`, подтвержденными правами `authorities`, очищенным паролем и флагом `isAuthenticated() == true`).
- **При ошибке:** Выбрасывается подкласс `AuthenticationException` (`BadCredentialsException`, `DisabledException`, `LockedException`).

## ProviderManager — реализация по умолчанию

В 99% случаев реализацией интерфейса выступает класс `ProviderManager`. Внутри него хранится обычный список провайдеров:

```
public class ProviderManager implements AuthenticationManager {  
    private List<AuthenticationProvider> providers;  
    // ...  
}
```

При вызове `authenticate()` он в цикле опрашивает каждый провайдер:

1. Вызывает `provider.supports(authentication.getClass())`.
2. Если провайдер возвращает `true`, вызывается `provider.authenticate(authentication)`.
3. Если проверка успешна, цикл завершается, и готовый объект возвращается вызывающей стороне.

### 3. Что такое AuthenticationProvider и какие виды бывают

Интерфейс `AuthenticationProvider` — это компонент, в котором реализована конкретная механика проверки учетных данных:

```
public interface AuthenticationProvider {  
    Authentication authenticate(Authentication authentication) throws AuthenticationException;  
    boolean supports(Class<?> authentication);  
}
```

| Провайдер                                     | Для чего применяется                                                     | С какими компонентами работает                                                            |
|-----------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>DaoAuthenticationProvider</code>        | Классический вход по логину и паролю из базы данных (наш проект).        | <code>UserDetailsService</code> (загрузка из БД) + <code>PasswordEncoder</code> (BCrypt). |
| <code>LdapAuthenticationProvider</code>       | Вход через корпоративный каталог Active Directory / LDAP.                | <code>LdapAuthenticator</code> , <code>LdapAuthoritiesPopulator</code> .                  |
| <code>JwtAuthenticationProvider</code>        | Проверка цифровой подписи и claims токенов JWT (OAuth2 Resource Server). | <code>JwtDecoder</code> , <code>JwtAuthenticationConverter</code> .                       |
| <code>RememberMeAuthenticationProvider</code> | Вход по долгоживущей cookie "Запомнить меня".                            | <code>RememberMeServices</code> , ключ подписи cookie.                                    |
| <code>AnonymousAuthenticationProvider</code>  | Предоставляет анонимный контекст неавторизованным гостям.                | Константа роли <code>ROLE_ANONYMOUS</code> .                                              |

### 4. Пошаговая анатомия работы DaoAuthenticationProvider (5 этапов)

Когда в контроллере вызывается `authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(login, password))`, провайдер `DaoAuthenticationProvider` выполняет строго 5 шагов:

#### 1. Шаг 1: Загрузка досье из БД ( `loadUserByUsername` )

Провайдер извлекает имя пользователя из токена и вызывает `userDetailsService.loadUserByUsername(username)`. Сервис идет в таблицу `users` через JPA Repository и возвращает объект `UserDetails` ( `AppPrincipal` ). Если пользователь не найден — сразу выбрасывается `UsernameNotFoundException`.

#### 2. Шаг 2: Проверка системных статусов аккаунта (Account Status Checks)

Провайдер проверяет 4 булевых флага:

- `isAccountNonLocked()` — не заблокирован ли пользователь? (если false \$to\$ `LockedException` );
- `isEnabled()` — активен ли пользователь? (если false \$to\$ `DisabledException` );

- `isAccountNonExpired()` — не истек ли срок действия учетной записи? (если false `$\to$ AccountExpiredException`);
- `isCredentialsNonExpired()` — не протух ли пароль? (если false `$\to$ CredentialsExpiredException`).

### 3. Шаг 3: Сверка хэша пароля ( `passwordEncoder.matches` )

Провайдер берет сырой пароль от клиента и хэш из `UserDetails.getPassword()`, после чего вызывает:

```
if (!passwordEncoder.matches(rawPassword, userDetails.getPassword())) {
    throw new BadCredentialsException("Неверный логин или пароль");
}
```

### 4. Шаг 4: Стирание пароля из памяти (Credentials Erasing)

В целях безопасности сырой пароль удаляется из объекта токена ( `credentials = null` ), чтобы он случайно не утек в логи, дампы кучи (Heap Dump) или трассировку стека.

### 5. Шаг 5: Создание подтвержденного объекта Authentication

Провайдер конструирует новый токен с тремя параметрами: `new`

`UsernamePasswordAuthenticationToken(principal, null, userDetails.getAuthorities())`. У этого токена флаг `isAuthenticated()` равен `true`.

## 5. Сквозная блок-схема процесса аутентификации



## 6. Зачем пишется отдельный метод @Bean для AuthenticationManager

Один из самых частых вопросов: «Зачем в `SecurityConfiguration` объявлять отдельный бин менеджера, если Spring Security и так уже настроен?».

### Проблема скрытого бина (NoSuchBeanDefinitionException)

По умолчанию Spring Security создает экземпляр `AuthenticationManager` внутри своих фильтров исключительно для внутренних нужд и **не экспортирует его в `ApplicationContext`** в качестве публичного Spring-бина.

Если в `AuthController` попытаться заинжектировать его через конструктор:

```
@RestController
@RequiredArgsConstructor
public class AuthController {
    // БЕЗ МЕТОДА @Bean БУДЕТ ОШИБКА ПРИ СТАРТЕ:
    // Parameter 0 of constructor in AuthController required a bean of type 'AuthenticationManager' that could not be found.
    private final AuthenticationManager authenticationManager;
}
```

## Вывод

Метод с аннотацией `@Bean` пишется ровно для того, чтобы **«выставить» менеджер наружу** в общий контейнер внедрения зависимостей (IoC Container), позволив внедрять его в любые контроллеры и сервисы.



## 7. Два канонических способа объявления бина в Spring Security 6

| Способ                                                                          | Фрагмент кода                                                                                                                                                                           | Как работает и когда применять                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Способ 1 (Рекомендуемый):</b> Через <code>AuthenticationConfiguration</code> | <pre>@Bean public AuthenticationManager authenticationManager(     AuthenticationConfiguration authConfig) throws Exception {     return authConfig.getAuthenticationManager(); }</pre> | <b>Стандарт Spring Boot 3.</b> Spring автоматически находит зарегистрированные в проекте бины <code>UserDetailsService</code> и <code>PasswordEncoder</code> , под капотом собирает <code>DaoAuthenticationProvider</code> и отдает готовый <code>ProviderManager</code> . |

| Способ                                                               | Фрагмент кода                                                                                                                                                                                                                                                                                                                                                                                | Как работает и когда применять                                                                                                                                                                                                    |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Способ 2 (Явный):</b> Ручная сборка<br><div>ProviderManager</div> | <pre> @Bean public AuthenticationManager authenticationManager(     UserDetailsServiceImpl userDetailsService,     PasswordEncoder passwordEncoder) {     DaoAuthenticationProvider provider = new DaoAuthenticationProvider();     provider.setUserDetailsService(userDetailsService);     provider.setPasswordEncoder(passwordEncoder);     return new ProviderManager(provider); } </pre> | <b>Полный контроль.</b> Применяется, когда в приложении несколько источников входа (например, база данных + корпоративный LDAP) и нужно передать список провайдеров <code>new ProviderManager(daoProvider, ldapProvider)</code> . |

## 8. Что скрывается внутри AuthenticationConfiguration.java

Если открыть исходный код класса `AuthenticationConfiguration.java` , можно увидеть, как фреймворк выполняет автоматическую связку:

```

@Bean
public static InitializeUserDetailsBeanManagerConfigurer initializeUserDetailsBeanManagerConfigurer
(...) {
    // Ищет в контексте бин типа UserDetailsService
    return new InitializeUserDetailsBeanManagerConfigurer(context);
}

@Bean
public static InitializeAuthenticationProviderBeanManagerConfigurer initializeAuthenticationProviderBe
anManagerConfigurer(...) {
    // Ищет PasswordEncoder и связывает его с UserDetailsService в DaoAuthenticationProvider
    return new InitializeAuthenticationProviderBeanManagerConfigurer(context);
}

```

## 9. Где искать в официальной документации Spring (docs.spring.io)

Официальный портал: [docs.spring.io/spring-security/reference/](https://docs.spring.io/spring-security/reference/).

### Критическое разделение: Authentication vs Authorization

- **Authentication (Аутентификация — /servlet/authentication/... )**: Отвечает на вопрос «Кто ты?». Ищи здесь всё про логин, пароли, токены, `AuthenticationManager`, `DaoAuthenticationProvider`, `UserDetailsService`, `PasswordEncoder`.
- **Authorization (Авторизация — /servlet/authorization/... )**: Отвечает на вопрос «Что тебе разрешено делать?». Ищи здесь всё про роли, права доступа, `@PreAuthorize`, `requestMatchers()`, ошибки 403 Forbidden.

## 10. Как смотреть документацию и исходники прямо в IntelliJ IDEA

---

В 1С нажатие `Ctrl + F1` открывает Синтакс-помощник. В Java документация устроена двухуровнево: скомпилированный байткод `.jar` и исходники с комментариями `-sources.jar`.

### 1. Горячая клавиша Quick Documentation ( `Ctrl + Q` / `F1` ):

Поставьте курсор на имя метода или класса (например, `getAuthenticationManager` ) и нажмите `Ctrl + Q` . IDEA откроет всплывающее окно с примерами кода из Javadoc.

### 2. Кнопка "Download Sources" (Скачать исходники):

Если при переходе в класс открывается декомпилированный код без комментариев, нажмите на желтой полосе сверху кнопку "**Download Sources**". Gradle скачает оригинальные Java-файлы с комментариями создателей фреймворка.

## 11. Как комфортно читать документацию на английском с уровнем A2+/B1

---

Технический английский в документации Spring сухой, структурированный и состоит из 300–400 повторяющихся терминов.

### 4-шаговая методика диагонального чтения

1. **Шаг 1: Смотрите на схемы и код (70% понимания).** Пролистайте страницу сразу до диаграммы или листинга Java. Названия методов и типов объясняют суть лучше любого текста.
2. **Шаг 2: Читайте информационные блоки (Callouts).** Блоки *Tip* (рекомендация), *Note* (важное примечание) и *Warning / Deprecated* (устаревший код) содержат квинтэссенцию страницы.
3. **Шаг 3: Читайте только первое предложение каждого абзаца.** В англоязычной технической литературе главное утверждение всегда идет в первом предложении (Topic Sentence). Если оно не про вашу проблему — пропускайте весь абзац.
4. **Шаг 4: Используйте двуязычный переводчик (Immersive Translate).** Расширение для браузера, выводящее русский перевод под оригинальной английской строкой и не ломающее код.

## 12. Микро-словарь терминов Spring Security

| Термин                   | Перевод                         | Пример в контексте Spring Security                                                      |
|--------------------------|---------------------------------|-----------------------------------------------------------------------------------------|
| <b>delegate</b>          | Делегировать / передавать       | <i>AuthenticationManager delegates check to AuthenticationProvider.</i>                 |
| <b>intercept</b>         | Перехватывать                   | <i>Filter intercepts HTTP request before Controller.</i>                                |
| <b>invoke</b>            | Вызывать / запускать            | <i>Invoke authenticationEntryPoint on 401 error.</i>                                    |
| <b>expose</b>            | Выставлять наружу / публиковать | <i>Expose AuthenticationManager as a Spring @Bean.</i>                                  |
| <b>override</b>          | Переопределять                  | <i>Override default security configuration.</i>                                         |
| <b>bypass</b>            | Пропускать в обход              | <i>Public URLs bypass JWT verification filter.</i>                                      |
| <b>validate / verify</b> | Проверять корректность          | <i>Verify signature and expiration of JWT token.</i>                                    |
| <b>populate</b>          | Заполнять данными               | <i>Populate SecurityContext with GrantedAuthorities.</i>                                |
| <b>persist</b>           | Сохранять в базу данных         | <i>Persist refresh token hash into PostgreSQL.</i>                                      |
| <b>extract</b>           | Извлекать                       | <i>Extract username claim from JWT payload.</i>                                         |
| <b>Principal</b>         | Субъект безопасности            | Объект вошедшего пользователя ( <code>UserDetails</code> / <code>AppPrincipal</code> ). |
| <b>Credentials</b>       | Реквизиты доступа               | Секретные данные для проверки (пароль, токен, сертификат).                              |
| <b>Authority</b>         | Полномочие / право              | Неделимое разрешение на действие ( <code>UNITS_READ</code> ).                           |
| <b>Prerequisite</b>      | Предварительное условие         | Что должно быть настроено до выполнения шага.                                           |
| <b>Stateless</b>         | Без сохранения состояния        | Сервер не держит HTTP-сессий (REST + JWT).                                              |
| <b>Downstream</b>        | Дальше по конвейеру             | Компоненты, вызываемые следующими в цепочке обработки.                                  |

### Памятка для быстрой самопроверки

- `AuthenticationManager` координирует провайдеров.
- `DaoAuthenticationProvider` связывает `UserDetailsService` и `PasswordEncoder` .
- `UserDetailsService` только читает из базы и не проверяет пароли.
- Бин `@Bean public AuthenticationManager` нужен для инжекта в контроллер.